

# INTERACTIVE CLI MUSIC PLAYER

## Project Report

### 1. Project Overview

The Interactive CLI Music Player is a terminal-based music player developed using Node.js. Unlike conventional command-line programs that display information line by line, this project provides a dynamic and interactive terminal interface that continuously updates in the same location.

The application acts as a terminal-based remote controller and visual interface for an external audio playback process. Users can navigate through a collection of songs, start and control playback, pause or resume music, move between tracks, and monitor playback progress through a real-time progress bar.

The project demonstrates how Node.js can combine terminal input handling, process management, asynchronous programming, file-system operations, and ANSI terminal control to create an interactive command-line application.

### 2. Problem Statement

Traditional command-line applications generally operate sequentially, where every new output is printed on a new line. This approach is not suitable for applications that require continuous interaction and real-time updates.

A music player requires the ability to accept keyboard input immediately, display a changing playback state, control an external media player, track the currently playing song, display playback progress, and update the terminal interface without continuously generating new lines.

Therefore, the objective of this project is to develop a fully interactive terminal-based music player capable of providing a smooth user experience while operating entirely through the command line.

### 3. Objectives

1. Develop an interactive music player using Node.js.
2. Implement real-time keyboard input without requiring the Enter key.
3. Control audio playback through an external media player.
4. Create a dynamic terminal user interface.
5. Implement song navigation using keyboard controls.
6. Display real-time playback progress.
7. Manage background processes efficiently.
8. Prevent multiple audio processes from playing simultaneously.
9. Demonstrate asynchronous programming and event-driven architecture in Node.js.

### 4. Technologies Used

Node.js — Main application runtime

JavaScript — Application logic

VLC Media Player — Background audio playback

child\_process.spawn — Starting and controlling external processes

fs module — Reading songs from the file system

process.stdin — Capturing keyboard input

ANSI Escape Codes — Dynamic terminal rendering

afinfo — Retrieving audio duration on macOS

## 5. System Architecture

The application follows an event-driven architecture in which user input, audio playback, process events, and UI updates work together.

Major components:

- File System Module — Scans the /songs directory and identifies available .mp3 files.
- Terminal Input Handler — Places the terminal in raw input mode so individual key presses can be detected immediately.
- State Manager — Maintains the selected song, cursor position, playback state, total duration, and elapsed time.
- VLC Process — Runs as a background process and performs actual audio playback.
- Duration Detector — Uses the macOS afinfo utility to determine audio duration.
- Terminal Renderer — Uses ANSI escape sequences to continuously redraw the interface.
- Progress Tracker — Updates elapsed playback time and calculates the current progress percentage.

## 6. Application Workflow

Step 1 — Initialization: Node.js reads the /songs directory and filters hidden or irrelevant files.

Step 2 — Display Song List: Available songs are displayed and a cursor indicates the selected track.

Step 3 — Capture Keyboard Input: process.stdin.setRawMode(true) enables immediate keypress detection.

Step 4 — Navigation: Arrow-key escape sequences are interpreted to update the cursor.

Step 5 — Start Playback: Enter starts the selected song after terminating previous VLC playback.

Step 6 — Determine Duration: afinfo obtains the selected audio file's duration.

Step 7 — Start VLC: VLC is launched in the background using child\_process.spawn().

Step 8 — Track Playback: A periodic timer updates elapsed playback time.

Step 9 — Update Terminal: ANSI escape sequences redraw the interface without filling the terminal with new lines.

## 7. Core Features

Song Navigation: Users navigate through available songs using the keyboard.

Play Song: Enter starts the selected track, while the previous VLC process is terminated to avoid overlapping audio.

Pause and Resume: Space sends a pause command directly to VLC through its standard input.

Next and Previous Track: N moves to the next song and D moves to the previous song.

Real-Time Progress Bar: A visual bar represents elapsed playback time versus total duration.

Dynamic Terminal Interface: ANSI escape codes allow the interface to be redrawn in place.

## 8. Raw Terminal Input

The application enables raw terminal input using `process.stdin.setRawMode(true)`. This allows individual keyboard events to be captured immediately rather than waiting for the user to press Enter.

Arrow keys are received as escape sequences and interpreted by the application as navigation commands. This technique is essential for creating a responsive terminal-based user interface.

## 9. ANSI Escape Codes

ANSI escape codes control the terminal cursor and screen. The application can move the cursor to a particular location and clear existing terminal content, allowing the interface to be redrawn repeatedly without filling the terminal with hundreds of lines of output.

This technique is used for displaying the song list, current selection, playback status, progress bars, and control instructions.

## 10. Background Process Management

Node.js provides the `child_process` module for interacting with external processes. The project uses `child_process.spawn()` to launch external commands.

VLC is responsible for actual music playback and operates in the background while Node.js manages the interface and controls. The macOS `afinfo` utility is used to obtain information about audio files, including their duration.

## 11. State Management

The application maintains state variables to synchronize the terminal interface with playback.

`cursor` stores the index of the currently selected song.

`isPaused` determines whether playback is paused.

`totalDuration` stores the duration of the current song.

`timeElapsed` tracks the approximate elapsed playback time.

Maintaining this state allows the application to render the correct playback information at any point.

## 12. Event-Driven Architecture

The project uses Node.js's event-driven programming model.

Keyboard input triggers navigation or playback controls. Enter starts a selected song, Space pauses or resumes playback, N and D change tracks, the timer interval updates playback progress, and VLC process events can be used to detect process termination.

This architecture allows the application to remain responsive while background operations are running.

## 13. Error Handling and Edge Cases

**Negative Progress / Progress Bar Crash:** When a song reaches its end, the timer may continue increasing `timeElapsed`. If elapsed time exceeds total duration, the remaining progress can become negative and cause a `RangeError`. The solution is to clamp progress between 0% and 100%, for example with `Math.min(1, timeElapsed / totalDuration)`.

**Lack of Automatic Next-Track Playback:** The current implementation does not automatically detect when VLC finishes a song. A future implementation can listen for the VLC process close event and start the next track.

**macOS Dependency:** The application relies on `afinfo` and a macOS-specific VLC path. Cross-platform support can be achieved by detecting the operating system, locating VLC dynamically, and using a cross-platform audio metadata library.

## 14. Advantages of the Project

Technical advantages include event-driven Node.js programming, asynchronous process management, real-time terminal interaction, inter-process communication, file-system operations, responsive command-line UI, external application integration, and state synchronization.

From a user-experience perspective, the application provides simple keyboard-based controls, requires no mouse, gives real-time playback feedback, maintains a clean terminal interface, and enables fast navigation.

## 15. Limitations

1. Primarily designed for macOS.
2. VLC must be installed.
3. The VLC executable path may be hardcoded.
4. Audio duration retrieval depends on `afinfo`.
5. Playback progress is estimated with a timer rather than VLC's exact position.
6. Automatic next-track playback is not yet implemented.
7. The interface is limited to terminal environments.
8. Supported audio formats depend on the playback system.

## 16. Future Enhancements

**Cross-Platform Support:** Detect the operating system and locate VLC dynamically.

**Automatic Playlist Playback:** Start the next song when the current track finishes.

**Accurate Playback Position:** Query VLC for the actual playback position instead of estimating with a timer.

**Volume Controls:** Add keyboard shortcuts for volume increase, decrease, and mute.

**Shuffle Mode:** Randomize playlist playback.

Repeat Mode: Repeat the current song or entire playlist.

Search Functionality: Search for songs instead of navigating through the entire list.

Playlist Management: Create, save, and load custom playlists.

Cross-Platform Audio Metadata: Replace `afinfo` with a Node.js audio metadata package.

## 17. Learning Outcomes

The project provides practical experience with Node.js programming, JavaScript event-driven architecture, file-system operations, terminal input processing, raw keyboard input, ANSI terminal control, child-process management, inter-process communication, asynchronous programming, timers and event loops, state management, error handling, and external application integration.

## 18. Conclusion

The Interactive CLI Music Player demonstrates how Node.js can be used to create a responsive and interactive application within a terminal environment.

The project combines raw keyboard input, ANSI terminal control, file-system operations, asynchronous timers, external process management, and VLC integration to provide music playback through a command-line interface.

A key achievement is transforming the traditionally static command-line environment into a continuously updating interface. Although the current implementation has limitations such as macOS-specific dependencies and the absence of automatic next-track playback, these provide clear opportunities for future development.

Overall, the project provides a practical demonstration of Node.js, process management, event-driven programming, terminal control, and system-level application integration.